

Universidad Internacional del Ecuador

Escuela de Ingeniería Automotriz



TALLER N ° 8

TÍTULO: Deber_8-P3

AUTOR: REYES LUCAS ANGELO GUILLERMO

**ASIGNATURA: LÓGICA DE PROGRAMACIÓN – PARALELO: 3 EIA-
Z**

TUTOR: DAVID HERNAN GUEVARA GUEVARA

**Guayaquil, Ecuador
Fecha: 19/08/2026**

Capture evidencia

Deber 3 - P3: Mejora analítica de mantenimientos

Complete las celdas marcadas con `TODO`. La solución debe usar archivos, pandas, NumPy, matplotlib, ciclos, condicionales, validaciones y manejo de errores.

```
[1]
✓ 0s
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

1. Carga de datos y manejo de errores

```
[3]
✓ 0s
def cargar_datos(ruta: str) -> pd.DataFrame | None:
    try:
        path = Path(ruta)
        if not path.exists():
            print(f"Error: El archivo '{ruta}' no existe.")
            return None

        df = pd.read_csv(ruta)

        if df.empty:
            print(f"Error: El archivo '{ruta}' está vacío.")
            return None

        return df
    except FileNotFoundError:
        print(f"Error: No se encontró el archivo '{ruta}'.")
    except pd.errors.EmptyDataError:
        print(f"Error: El archivo '{ruta}' está vacío o corrupto.")
    except pd.errors.ParserError:
        print(f"Error: El formato del CSV no es interpretable.")
    except Exception as e:
        print(f"Error inesperado: {e}")
    return None
```

Deber_3_P3.ipynb ☆ Guardando...

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Comandos + Código + Texto Ejecutar todo

Archivos

- sample_data
- mantenimientos.csv
- mantenimientos_con_errores.csv

```
[3]
✓ 0s
print(f"Error: El formato del CSV no es interpretable.")
except Exception as e:
    print(f"Error inesperado: {e}")
return None

[9]
✓ 0s
# PRUEBA 1: archivo válido
df = cargar_datos("mantenimientos.csv")
print(f"Archivo 'mantenimientos.csv' cargado con éxito. Total de filas: {len(df)}")

Archivo 'mantenimientos.csv' cargado con éxito. Total de filas: 120

[10]
✓ 0s
# PRUEBA 2: archivo inexistente
_ = cargar_datos("archivo_que_no_existe.csv")

Error: El archivo 'archivo_que_no_existe.csv' no existe.
```

2. Validación y limpieza

2. Validación y limpieza

```
[11]
✓ 0s def validar_y_limpiar(df: pd.DataFrame) -> pd.DataFrame:
    codigos_vistos = set()
    filas_validas = []

    for idx, fila in df.iterrows():
        # 1. Código vacío
        if pd.isna(fila['código']) or str(fila['código']).strip() == "":
            continue

        # 2. Código duplicado
        codigo = fila['código']
        if codigo in codigos_vistos:
            continue

        # 3. Fecha inválida
        fecha_convertida = pd.to_datetime(fila['fecha'], errors='coerce')
        if pd.isna(fecha_convertida):
            continue

        # 4. Costo no numérico o negativo
        costo = pd.to_numeric(fila['costo'], errors='coerce')
        if pd.isna(costo) or costo < 0:
            continue
```

```
[12]
✓ 0s def validar_y_limpiar(df: pd.DataFrame) -> pd.DataFrame:
    # Copiamos el dataframe y estandarizamos los nombres de las columnas (sin tildes y minúsculas)
    df = df.copy()
    df.columns = (df.columns.str.lower()
                  .str.strip()
                  .str.replace('á', 'a')
                  .str.replace('é', 'e')
                  .str.replace('í', 'i')
                  .str.replace('ó', 'o')
                  .str.replace('ú', 'u'))

    codigos_vistos = set()
    filas_validas = []

    for idx, fila in df.iterrows():
        # 1. Código vacío
        if pd.isna(fila['codigo']) or str(fila['codigo']).strip() == "":
            continue

        # 2. Código duplicado
        codigo = fila['codigo']
        if codigo in codigos_vistos:
            continue

        # 3. Fecha inválida
```

```
[13]
✓ 0s print(f"Media de costo: {media_costo:.2f}")
print(f"Mediana de costo: {mediana_costo:.2f}")
print(f"Percentil 75 de costo: {p75_costo:.2f}")

# 4. Clasificación con np.where()
df_limpio['clasificacion_costo'] = np.where(
    costos <= mediana_costo, 'Económico',
    np.where(costos <= p75_costo, 'Moderado', 'Alto')
)

print("\nMuestra de datos clasificados con NumPy:")
print(df_limpio[['codigo', 'costo', 'clasificacion_costo']].head())
```

```
...
Media de costo: 83.43
Mediana de costo: 83.21
Percentil 75 de costo: 113.80
```

```
Muestra de datos clasificados con NumPy:
   codigo  costo clasificacion_costo
0  MNT-0001  56.64      Económico
1  MNT-0002  53.86      Económico
2  MNT-0003  83.42      Moderado
3  MNT-0004  79.54      Económico
4      NaN  117.38      Alto
```

4. Análisis con pandas

```

# 1. Métricas principales usando pandas
num_registros_validos = len(df_limpio)
costo_total = df_limpio['costo'].sum()
costo_promedio = df_limpio['costo'].mean()

# Mantenimiento más costoso y más económico
idx_max = df_limpio['costo'].idxmax()
idx_min = df_limpio['costo'].idxmin()

mantenimiento_mas_costoso = df_limpio.loc[idx_max]
mantenimiento_mas_economico = df_limpio.loc[idx_min]

print(f"Número de registros válidos: {num_registros_validos}")
print(f"Costo total: ${costo_total:.2f}")
print(f"Costo promedio: ${costo_promedio:.2f}")
print(f"Mantenimiento más costoso (Código: {mantenimiento_mas_costoso.get('codigo', 'N/A')}): ${mantenimiento_mas_costoso['costo']:.2f}")
print(f"Mantenimiento más económico (Código: {mantenimiento_mas_economico.get('codigo', 'N/A')}): ${mantenimiento_mas_economico['costo']:.2f}")

# 2. Agrupación por tipo de equipo
if 'equipo' in df_limpio.columns:
    resumen_equipo = df_limpio.groupby('equipo')['costo'].agg(['count', 'mean', 'sum']).reset_index()
    resumen_equipo.columns = ['Equipo', 'Cantidad', 'Costo Promedio', 'Costo Total']
    print(f"\nResumen por Equipo:")
    print(resumen_equipo)

```

Número de registros válidos: 119
Costo total: \$9927.80
Costo promedio: \$83.43

Mantenimiento más costoso (Código: MNT-0119): \$172.63
Mantenimiento más económico (Código: MNT-0095): \$-50.00

Resumen por Equipo:

	Equipo	Cantidad	Costo Promedio	Costo Total
0	Desktop	19	89.773158	1705.69
1	Impresora	17	65.169412	1107.88
2	Laptop	23	86.186522	1982.29
3	Proyector	24	109.688333	2638.68
4	Router	14	58.838714	823.63

5. Visualización con matplotlib

```

# Gráfico 1: Costo total por área
resumen_area = df_limpio.groupby('area')['costo'].sum()
resumen_area.plot(kind='bar', color='salmon', title='Costo Total por Área')
plt.ylabel('Costo Total ($)')
plt.show()

```

area	Costo Total (\$)
FADCOM	1982.29
FCNIM	1107.88
FCSH	1705.69
FIEC	823.63
FIMCP	2638.68
GTSI	1982.29

